Exam

Dylan Assinner

May 2026

Introduction

Fission is a well studied process used in nuclear physics for the construction of nuclear power plants and nuclear weapons. One of the elements most commonly used for fission reactions is Uranium. It has two major isotopes: Uranium-238 and Uranium-235. Uranium-238 (^{238}U) can be found in nature and is a radioactive element, producing α-type radiation with a half-life of $4.463 \cdot 10^9$ years [2]. There is also the more radioactive isotope uranium-235 (^{235}U). This isotope is more favorable for fission.

The fission reaction we are going to study is fission using ^{235}U. If we add a neutron to this core, it transitions into the very unstable isotope of Uranium-236. This can then split into two or more lighter elements and an excess of neutrons. There are two prominent reactions here:

$$^1_0\text{n} + {}^{235}_{92}\text{U} \rightarrow {}^{236}_{92}\text{U} \rightarrow {}^{140}_{54}\text{Xe} + {}^{94}_{38}\text{Kr} + 2\ {}^1_0\text{n}$$

$$^1_0\text{n} + {}^{235}_{92}\text{U} \rightarrow {}^{236}_{92}\text{U} \rightarrow {}^{141}_{56}\text{Ba} + {}^{92}_{36}\text{Kr} + 3\ {}^1_0\text{n}$$

In this simulation, we will use the first reaction as a basis.

The intent of the simulation is to calculate the critical mass. This is when the reaction becomes self-sustaining.

In order to explain the concept of critical mass, we need to look at the environment in which the fission occurs. We will assume that there are no possibilities for fission outside our simulated mass (which will consist entirely of 235U and will always follow the first reaction, for simplicity). We also assume that every neutron produced will travel a short random distance. If it is still inside the mass at that point, it will provoke another fission reaction.

The critical mass is thus the mass for which an equal amount of neutrons stay inside the mass as are leaving the object, thus maintaining a constant steady amount of reactions. If the mass is greater than, the critical mass, it will become what we call “supercritical”, meaning that more fission reactions are initiated than originally started. If the mass is lower than the critical mass, the amount of fission reactions will diminish and eventually stop.

1 Implementation

The math for this project will be split into three parts:

1. Calculating the survivor fraction
2. coordinate randomization
3. mass distribution

1.1 calculating the survivor fraction

The survivor fraction is how we measure whether the Uranium has reached the critical mass. It is calculated by dividing the previous "generation" of fission by the current "generation" of fission.
If the survivor fraction (f) is lower than 1, we can conclude that we are below the critical mass, meaning that the reaction will die out.
If the survivor fraction is higher than 1, we can conclude that the reaction has gone supercritical.
By changing the mass (user input), we can try to find the point at which the survivor fraction is 1. To make sure this is accurate, we will allow the user to insert an amount of points (N) that need to be simulated. we will then simulate those N random points inside the volume. this will be our first generation. We will then simulate fission for each of those N points. If we check whether the neutrons are inside the volume, we can calculate the survivor fraction with the formula:

$$f = \frac{N_{\text{inside volume}}}{N_{\text{gen 1}}} \tag{1}$$

To be able to check whether the neutrons are inside the volume, we first need to build a randomizer for the movement of the neutrons inside the simulations.

1.2 coordinate randomization

Using a randomizer to generate random uniformly distributed points, we can calculate the path of the neutrons. If the neutrons have starting coordinates (x_0, y_0, z_0), we can describe their endpoints (x_1, y_1, z_1) using the equations:

$$\begin{cases} x_1 = x_0 + d\ \sin\theta\ \cos\phi \\ y_1 = y_0 + d\ \sin\theta\ \sin\phi \\ z_1 = z_0 + d\ \cos\theta \end{cases}$$

Here, d is the distance the neutron travels and θ and ϕ are the angles of spherical coordinates. We will need to randomize these three variables.

d Is a random variable between 1 and 0. ϕ Is a random variable between 2π and 0. Both of these variables are uniformly distributed.
θ is a bit trickier, for $\cos\theta$ is the uniformly distributed variable. We will thus have to randomize a value between -1 and 1 and use the arccos function to calculate the angle θ.

1.3 mass distribution

For the mass distribution, we have to take into account that we have two possible shapes to simulate: a beam (or cube) and a sphere.

1.3.1 beam

For the beam, we will assume that the mass (M) is equal to the volume (V). We can then change the shape of the beam using a scale factor (S). The influence of the scale factor can be shown with the following equations[3]:

$$M = V = a^2 b = \frac{a^2}{S} = b^3 S^2$$

Here, a is the width and the height of the beam and b is the depth of the beam. If we want to calculate a and b, we can derive following equations[3]:

$$a = \sqrt[3]{M \cdot S} \tag{2}$$

$$b = \sqrt[3]{\frac{M}{S^2}} \tag{3}$$

We can thus see that for $S = 1$, we get a perfect square.
We will assume that this beam is centered around (0, 0, 0). We can then create a starting point randomizer, using a uniform distribution between the following endpoints:

$$\begin{cases} x & \in \left[-\frac{a}{2}, \frac{a}{2}\right] \\ y & \in \left[-\frac{a}{2}, \frac{a}{2}\right] \\ z & \in \left[-\frac{b}{2}, \frac{b}{2}\right] \end{cases}$$

We can also use these conditions for the evaluation to check whether a point is inside the beam object.

1.3.2 sphere

For the sphere, we will also assume that the mass is equal to the volume. For this class, we don't need to implement a scale factor. We do however need to calculate the radius. this can be done by using the formula:

$$V = \frac{4}{3}\pi\, r^3$$

If we then substitute V for M, we can derive the formula for the radius:

$$r = \sqrt[3]{\frac{3}{4}\,M} \tag{4}$$

For the random starting points inside the sphere, we can reuse the code for coordinates of a neutron. Because the distance for the neutrons is between 0 and 1, we can simply multiply this number with the radius to get the desired output.
To evaluate whether a point is inside this sphere, we calculate the distance from the center of the circle (0, 0, 0) and compare this to the radius.

2 Flowcharts

To get a clear view of how the code works, we will use flowcharts of the classes , but first we will explain the way the files are organized:

- **particle.py**
  contains the Particle parent class and the Uranium and Neutron subclasses.
- **shapes.py**
  contains the implementation of the Simulator parent class with the beam (BeamSim) and sphere (SphereSim) subclasses. It also contains an unused Point class.
- **test.py**
  contains the script with the explanation and implementation of the code for the tasks[3].
- **randomizer.py**
  testing file for the randomizer implemented in the Neutron SphereSim classes.
- **color_picker.md**
  markdown file containing the hexcodes with the colors used in visual simulations.

All these files, along with the files of this report are available through the github repository:

3 Behind the code

4 results

5 Conclusion

References

[1] Wikipedia contributors. Uranium, 4 2026.

[2] Wikipedia contributors. Uranium-238, 4 2026.

[3] Nick Van Remortel and Federico De Lillo. *Examen Eerste Zittijd 2026 Inleiding programmeren voor fysici*. 2026.